

## Threat Hunting and Investigative Skills

### Module 1 — Core Investigation Skills (Capture-the-Flag Writeup)

**Tool:** Microsoft Defender Advanced Hunting (KQL) **Ticket:** Help Desk #4451, received 22 April 2026, 09:14 UTC **Reporter:** Mark Smith, Finance **Affected host:** NPT-WS01 **Reported symptom:** "My machine was throwing login prompts at me through the night... seems fine this morning." **Hunt window used:** 2026-04-22T02:00:00Z to 2026-04-22T08:00:00Z (widened from the ticket's 04:30–06:00 focus window to catch setup/cleanup activity just outside it)

This writeup documents an 11-flag threat hunt run against a single compromised workstation. Every query below is the real query used, followed by a plain breakdown of what each line does and why, aimed at someone new to KQL. The story it tells: a "quiet" RDP prompt was actually a successful remote logon using a stolen helpdesk credential, followed by remote code execution, a C2 implant, three separate persistence mechanisms, a new backdoor admin account, and lateral movement toward a second host.

Every query in this hunt starts with the same three lines, so it's shown once here instead of every time:

```
let start_time = datetime(2026-04-22T02:00:00.00Z);
```

```
let end_time = datetime(2026-04-22T08:00:00.00Z);
```

```
let HostInQuestion = "npt-ws01";
```

let just creates a named variable you can reuse later in the query instead of retyping it. `datetime(...)` builds a timestamp — the format is YYYY-MM-DDTHH:mm:ssZ, where the T separates date from time and the Z means UTC. So this block says: "only look at April 22, 2026, between 2:00 AM and 8:00 AM UTC, on the machine named npt-ws01." Every query after this filters `TimeGenerated` between `(start_time .. end_time)` and `DeviceName == HostInQuestion` first, before asking anything more specific — that's standard hunting hygiene: narrow the haystack before you look for the needle.

## **Flag 1 — The Logon**

**Skill:** Recognising a suspicious authentication event. **Table/column:** DeviceLogonEvents → AccountName **Flag:** helpdesk

DeviceLogonEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where ActionType == "LogonSuccess"

DeviceLogonEvents is the table that records every logon attempt Defender sees on the endpoint. where ActionType == "LogonSuccess" cuts out all the failed attempts (the "login prompts all night" Mark ignored) and leaves only the logons that actually succeeded. Scanning the results, one account stands out: helpdesk. Mark's own account never appears as the successful logon — someone else got in using a shared/service-style credential that has no reason to be logging into a Finance workstation.

## **Flag 2 — The Source**

**Skill:** Tying a logon to an external IP. **Table/column:** DeviceLogonEvents → RemoteIP **Flag:** 20.110.92.50

DeviceLogonEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where ActionType == "LogonSuccess"

| project TimeGenerated, DeviceName, ActionType, AccountName, RemoteIP

Same query as Flag 1, but with project added — project just picks which columns to display, in whatever order you list them, so you're not scrolling through dozens of irrelevant fields. Reading across the helpdesk row to RemoteIP shows 20.110.92.50 — a public internet address, not an

internal corporate IP. A "helpdesk" account authenticating interactively from the open internet is the actual red flag here; legitimate helpdesk logons come from inside the network.

### **Flag 3 — The Process**

**Skill:** Process execution / command-line analysis. **Table/column:** DeviceProcessEvents → ProcessCommandLine **Flag:** cmd.exe /Q /c start "" "C:\Windows\Temp\WindowsUpdate.exe" 1>\Windows\Temp\uYgvso 2>&1

DeviceProcessEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where AccountName == "helpdesk"

| project TimeGenerated, ActionType, FileName, DeviceName, ProcessCommandLine, InitiatingProcessCommandLine

DeviceProcessEvents logs every process that runs on the machine. Filtering to AccountName == "helpdesk" shows only what ran under the compromised account, and one line jumps out: a cmd.exe /Q /c start command silently launching WindowsUpdate.exe out of C:\Windows\Temp\. That's not really a Windows Update binary — it's malware named to blend in. The /Q /c start pattern (quiet, run-and-return) plus redirecting output to a random-looking file (uYgvso) is the textbook fingerprint of Impacket's wmiexec tool, a common attacker tool for remote command execution.

### **Flag 4 — The Tree**

**Skill:** Parent-child process relationships. **Table/column:** DeviceProcessEvents → InitiatingProcessFileName **Flag:** wmiprvse.exe

DeviceProcessEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where AccountName == "helpdesk"

| project TimeGenerated, ActionType, FileName, DeviceName, ProcessCommandLine, InitiatingProcessCommandLine, InitiatingProcessFileName

Adding InitiatingProcessFileName to the projection shows what actually spawned that cmd.exe — and it's wmiexec.exe, the WMI Provider Host. Normally cmd.exe gets launched by a user double-clicking something or a script; here it was launched by the WMI subsystem itself. That confirms this wasn't local user activity — it's remote code execution over WMI, which is exactly how wmiexec works: attacker sends a command over the network, WMI executes it on the target with no interactive session needed.

### **Flag 5 — The Connection**

**Skill:** Outbound C2 identification; filtering out legitimate Microsoft traffic. **Table/column:**

DeviceNetworkEvents → RemoteUrl **Flag:** updates.abordasync.website

DeviceNetworkEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where InitiatingProcessAccountName == "helpdesk"

| where isnotempty(RemoteUrl)

| project TimeGenerated, ActionType, DeviceName, RemoteUrl, InitiatingProcessCommandLine, InitiatingProcessFileName

isnotempty(RemoteUrl) just drops rows where that field is blank, so you're only looking at connections that actually resolved to a named domain. Filtering to the helpdesk account's network activity and skimming past normal \*.microsoft.com / \*.windows.com traffic, one domain doesn't belong: updates.abordasync.website. It resolves back to the same 20.110.92.50 from Flag 2 — same infrastructure the attacker logged in from is now being used as the command-and-control (C2) destination the implant is calling home to.

### **Flag 6 — The File**

**Skill:** File drops in staging locations; network-delivered file attribution. **Table/column:**

DeviceFileEvents → SHA256 **Flag:**

20cef6a013953890f9d38605d25d60dd63b42b09946bbb18ddb4a456da306e77

DeviceFileEvents

| where isnotempty(SHA256)

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where ActionType == "FileCreated"

| where FileName ends with ".exe"

| project TimeGenerated, ActionType, DeviceName, SHA256, InitiatingProcessFileName, FileName, InitiatingProcessCommandLine, RequestAccountName

| order by TimeGenerated asc

This finds the moment WindowsUpdate.exe first landed in C:\Windows\Temp\ and captures its SHA256 hash — the unique fingerprint you'd use to block this exact file everywhere else in the environment, or check it against threat intel feeds. order by TimeGenerated asc just sorts oldest-first so you see the drop before the execution.

The interesting detail here is InitiatingProcessFileName, which shows ntoskrnl.exe — the Windows kernel image itself, not a normal user process. That happens when a file is written to disk over an SMB network share (like ADMIN\$) rather than by something running locally: the kernel's SMB server component performs the write, so Defender attributes it to the kernel rather than to a user-mode program. In plain terms — this confirms the file wasn't downloaded by a browser or dropped by a local script; it was pushed onto the box remotely over the network. That matches perfectly with wmiexec's normal behavior: copy the payload over SMB first, then trigger it remotely via WMI (which is exactly what Flags 3–4 showed).

### **Flag 7 — The Key**

**Skill:** Registry Run-key persistence. **Table/column:** DeviceRegistryEvents → RegistryValueName

**Flag:** WindowsHealthCheck

DeviceRegistryEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where RegistryKey contains "run"

| where isnotempty(RegistryValueName)

| project Timestamp, ActionType, DeviceName, RegistryKey, RegistryValueData, RegistryValueName, RegistryValueType

RegistryKey contains "run" catches writes to ...\\CurrentVersion\\Run, the classic autostart location — anything listed there launches automatically every time the machine boots or the user logs on. The value name WindowsHealthCheck is designed to look like a legitimate system task, but RegistryValueData points straight back at the implant sitting in Temp. This is persistence mechanism #1: even if the original process gets killed, a reboot brings the malware back.

### **Flag 8 — The Task**

**Skill:** Scheduled-task persistence. **Table/column:** DeviceProcessEvents → ProcessCommandLine (/tn value) **Flag:** GoogleUpdaterTask

DeviceProcessEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where isnotempty(ProcessCommandLine)

| where ProcessCommandLine contains "Updater"

| project Timestamp, ActionType, DeviceName, ProcessCommandLine, InitiatingProcessCommandLine

This looks for a schtasks.exe /create command. The /tn (task name) argument is GoogleUpdaterTask — impersonating Google's real updater to avoid a second glance from an admin scanning the Task Scheduler. This is persistence mechanism #2, running independently of the Run key, so removing one alone wouldn't fully evict the attacker.

### **Flag 9 — The Service**

**Skill:** Service-based persistence. **Table/column:** DeviceProcessEvents → ProcessCommandLine (sc.exe create) **Flag:** WindowsHealthSvc

DeviceProcessEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where isnotempty(ProcessCommandLine)

| where ProcessCommandLine contains "sc.exe create"

| project Timestamp, ActionType, DeviceName, ProcessCommandLine,  
InitiatingProcessCommandLine

sc.exe creates a new Windows service. Both the service name and its display name masquerade as a legitimate health-check service, and its binPath argument points at the same implant in Temp. Windows services can run with SYSTEM-level privileges and restart automatically, which makes this persistence mechanism #3 — and the most privileged of the three.

### **Flag 10 — The Account**

**Skill:** Local account creation and privilege escalation. **Table/column:** DeviceProcessEvents → ProcessCommandLine (net user ... /add) **Flag:** nexus\_admin

DeviceProcessEvents

| where TimeGenerated between (start\_time .. end\_time)

| where DeviceName == HostInQuestion

| where isnotempty(ProcessCommandLine)

| where ProcessCommandLine contains "net user"

| project Timestamp, ActionType, DeviceName, ProcessCommandLine,  
InitiatingProcessCommandLine

Two net.exe/net1.exe commands land seconds apart: one creates a brand-new local account (nexus\_admin), the next adds it to the local Administrators group. This is the attacker planting their own backdoor admin identity on the box — a credential that doesn't depend on the original helpdesk compromise being noticed or reset, and gives full local admin rights going forward.

## **Flag 11 — The Endpoint**

**Skill:** Scoping an investigation across multiple hosts. **Table/column:** AlertEvidence / DeviceInfo → DeviceName **Flag:** npt-srv01

AlertEvidence

| where Timestamp between (start\_time .. end\_time)

| where DeviceName startswith "npt-"

| where DeviceName != "npt-ws01"

| summarize AlertCount = dcount(AlertId), EvidenceRoles = make\_set(EvidenceRole) by DeviceName

| order by AlertCount desc

DeviceInfo

| where DeviceName in ("npt-c2-01", "npt-linux01", "npt-srv01")

| where Timestamp between (start\_time .. end\_time)

| summarize arg\_max(Timestamp, OnboardingStatus, DeviceType, OSPlatform) by DeviceName

The first query checks every other npt- machine for alert activity in the window: dcount(AlertId) counts how many distinct alerts mention each device, and make\_set(EvidenceRole) shows what role that device played in those alerts (e.g. attacker vs. impacted). npt-srv01 comes out on top. The second query is a sanity check — arg\_max(Timestamp, ...) grabs the most recent status row for each candidate host to confirm it's a real, onboarded machine and not noise, ruling out npt-c2-01 and npt-linux01. Conclusion: the attacker didn't stop at the workstation — they moved laterally toward npt-srv01, so containment has to extend there too.



## **D1 — Containment Plan**

### **Isolate immediately:**

- NPT-WS01 — isolate from the network via Defender's device isolation (or physically pull the NIC) to stop further C2 traffic and lateral movement from the source host.
- NPT-SRV01 — isolate as well, or at minimum place under enhanced monitoring, since it's the confirmed lateral-movement target; treat it as compromised until proven otherwise, not just "at risk."

### **Accounts to disable/reset:**

- helpdesk — disable immediately and force a credential reset. This account is the initial foothold; if it's a shared/service account, audit everywhere else it has access, since the attacker may have reused it beyond NPT-WS01.
- nexus\_admin — delete this account outright. It's an attacker-created backdoor with no legitimate purpose; don't just disable it, remove it and remove it from the local Administrators group if disabling and deletion don't happen atomically.
- Any account with cached credentials or an active session on NPT-WS01 during the window should be treated as potentially exposed and reset out of caution.

### **IOCs to block (network/endpoint-wide, not just on the two known hosts):**

- IP 20.110.92.50 — block outbound/inbound at the firewall and proxy; this is both the attacker's logon source and the C2 IP.
- Domain updates.abordasync.website — block at DNS/web proxy; add to threat intel blocklist.
- File hash 20cef6a013953890f9d38605d25d60dd63b42b09946bbb18ddb4a456da306e77 (WindowsUpdate.exe) — push to EDR/AV as a block/quarantine rule across the fleet, in case it was staged elsewhere.

### **Persistence to remove (all three, on NPT-WS01 and re-check on NPT-SRV01):**

1. Registry Run key WindowsHealthCheck under HKCU\...\CurrentVersion\Run (or HKLM, depending on where it landed) — delete the value.
2. Scheduled task GoogleUpdaterTask — delete via Task Scheduler / schtasks /delete.
3. Windows service WindowsHealthSvc — stop and delete the service (sc.exe delete), then remove the underlying binary.
4. The dropped implant itself, C:\Windows\Temp\WindowsUpdate.exe, plus its output artifact C:\Windows\Temp\uYgvso — delete both after forensic imaging/hash capture is complete.

### **Remediation follow-up:**

- Rebuild or fully re-image NPT-WS01 rather than trusting a manual cleanup, given three independent persistence mechanisms were found — assume there may be a fourth not yet discovered.
- Run the same 11-flag hunt pattern against NPT-SRV01 and the rest of the npt- fleet to confirm scope hasn't extended further.
- Review how the helpdesk account's credentials were obtained (phishing, credential stuffing, reused password) to close the actual entry vector, not just this instance of it.

## **D2 — Investigation Report**

**What happened:** A workstation belonging to a Finance user (NPT-WS01) was compromised via a network logon using a helpdesk account from external IP 20.110.92.50. The attacker used WMI-based remote execution (wmiexec) to run commands under that account, staged an implant (WindowsUpdate.exe) into C:\Windows\Temp\ by writing it over SMB, and established a C2 channel to updates.abordasync.website. To survive a reboot or process kill, the attacker installed three independent persistence mechanisms: a registry Run key, a scheduled task, and a Windows service. They then created a new local account (nexus\_admin) and escalated it to local Administrators, and began moving laterally toward a second host, NPT-SRV01.

## Timeline (22 April 2026, UTC):

1. Attacker authenticates to NPT-WS01 as helpdesk from 20.110.92.50 (network logon, LogonType 3) — this is the intrusion's start and the source of Mark's overnight login prompts.
2. Under the helpdesk session, wmiprvse.exe spawns cmd.exe, which silently launches WindowsUpdate.exe from Temp — remote code execution via WMI.
3. WindowsUpdate.exe (SHA256 20cef6a0...da306e77) is written to C:\Windows\Temp\ via an SMB-attributed kernel write (ntoskrnl.exe as initiating process) — confirming it arrived over the network rather than locally.
4. The implant beacons outbound to updates.abordasync.website, resolving to the same 20.110.92.50 — C2 established.
5. Three persistence mechanisms are installed in quick succession: Run key WindowsHealthCheck, scheduled task GoogleUpdaterTask, and service WindowsHealthSvc.
6. Local account nexus\_admin is created and added to Administrators — privilege escalation and backdoor access secured.
7. Alert evidence shows activity extending to NPT-SRV01 — lateral movement attempt/success beyond the original host.

**Impact:** Confirmed compromise of one Finance workstation with full remote-command capability, a persistent C2 implant, a privileged backdoor account, and evidence of movement toward at least one additional internal host. Given the account used had helpdesk-level access, the blast radius could extend to any system that account could reach — that scope isn't fully known without an environment-wide account-usage review.

**Scope:** Confirmed — NPT-WS01 (source/initial compromise) and NPT-SRV01 (lateral movement target). Not yet ruled out — any other host the helpdesk or nexus\_admin accounts could reach; this should be the immediate next hunting pass.

**Why this detection point:** The chosen anchor for this investigation is Flag 1 — the successful helpdesk network logon in DeviceLogonEvents. It's the earliest point in the entire chain, upstream of the execution, the C2, all three persistence mechanisms, and the lateral movement — every later flag is a downstream consequence of this one event. It's also the highest-confidence anomaly with the lowest false-positive risk: a shared/service-style account authenticating interactively from a public IP

has essentially no legitimate justification, unlike, say, a single scheduled-task execution which could plausibly be benign on its own. Anchoring here means an analyst monitoring authentication events alone — without needing to already suspect malware — could have caught this before any of the persistence or privilege escalation occurred, which is the whole point of picking a detection point: it should be the earliest, cheapest place to have stopped the incident.